

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
**«ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ
ФРАКТАЛЬНОГО ПЛАМЕНИ»**

БГУИР КП 6-05 0612 02 26 ПЗ

Выполнил студент группы 453501
СКРОБОТ Денис Алексеевич

(подпись студента)

Курсовой проект представлен на
проверку _____._____.2026

(подпись студента)

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ аналогов	6
1.1 Apophysis	6
1.2 JWildfire	7
1.3 Chaotica	8
1.4 Сравнительный анализ	9
2 Формирование функциональных требований	9
3 Анализ литературы	10
3.1 Принципы объектно-ориентированного проектирования	10
3.2 Принципы SOLID и качество кода	11
3.3 Шаблоны проектирования	11
3.4 Моделирование предметной области	11
3.5 Слоистая организация приложения	12
3.6 Контракты и переносимость моделей	12
4 Проектирование системы	13
4.1 Общая архитектура	13
4.2 Внешний API и роль gateway	14
4.3 Внутренние gRPC-контракты	15
4.4 Проектирование пользовательского интерфейса	15
4.5 Проектирование generator	16
4.6 Проектирование IAM	16
4.7 Модель данных generator	16
4.8 Модель данных IAM	17
4.9 Хранение изображений	17
4.10 Развёртывание	17
4.11 Язык программирования Java	18
4.12 Фреймворк Spring	19
4.13 Язык Go	20
4.14 Frontend: TypeScript, Vue, Tailwind CSS и Axios	20
4.15 Система сборки Gradle	21
4.16 База данных PostgreSQL	21
4.17 MinIO и S3-совместимое хранилище	21
4.18 gRPC	22
4.19 JWT	22
4.20 Контейнеризация Docker и Docker Compose	22
4.21 Среда разработки Visual Studio Code	23
4.22 Хостинг и развёртывание приложения	24
5 Тестирование программного обеспечения	25
5.1 Стратегия тестирования	25
5.2 Сценарии тестирования	26

Заключение 27

Список использованных источников 29

ВВЕДЕНИЕ

Фрактальная графика относится к области визуализации сложных математических моделей и активно используется в цифровом искусстве, дизайне и научной визуализации. Одним из наиболее выразительных направлений является генерация фрактального «пламени», предложенная С. Дрейвсом, где изображение формируется итеративными нелинейными преобразованиями и последующей тональной обработкой плотности [1]. Благодаря богатству форм и высокой детализации такие изображения востребованы как в творческих проектах, так и в обучении математическому моделированию.

Современные инструменты генерации фракталов чаще представлены настольными приложениями и ориентированы на локальную работу пользователя. При этом растёт спрос на веб-сервисы, обеспечивающие доступ с разных устройств, хранение истории и возможность публикации результатов. Для решения этих задач требуется система, сочетающая удобный пользовательский интерфейс, вычислительно ёмкий рендеринг и надёжное хранение артефактов.

Целью курсового проекта является создание распределённой системы генерации фракталов с веб-интерфейсом, поддержкой аутентификации пользователей и хранением истории созданных работ. Архитектура системы построена на принципах микросервисов, что облегчает масштабирование и независимое развитие компонентов [2]. Разработка выполнена в соответствии с принципами объектно-ориентированного проектирования и ориентирована на практическое применение современных инструментов сборки и развёртывания.

Для достижения поставленной цели были решены следующие задачи:

- анализ существующих решений и выявление их сильных и слабых сторон;
- формирование функциональных и нефункциональных требований к системе;
- проектирование микросервисной архитектуры и модели данных;
- выбор технологий реализации, сборки и развёртывания;
- разработка и тестирование ключевых сценариев работы пользователей.

1 Анализ аналогов

Для определения требований и позиционирования проекта были рассмотрены наиболее известные решения в области генерации фрактальных пламен. Сравнение выполнено по критериям: доступность (desktop/web), качество рендеринга, удобство настройки параметров, наличие истории и механизмов публикации результатов.

1.1 Apophysis

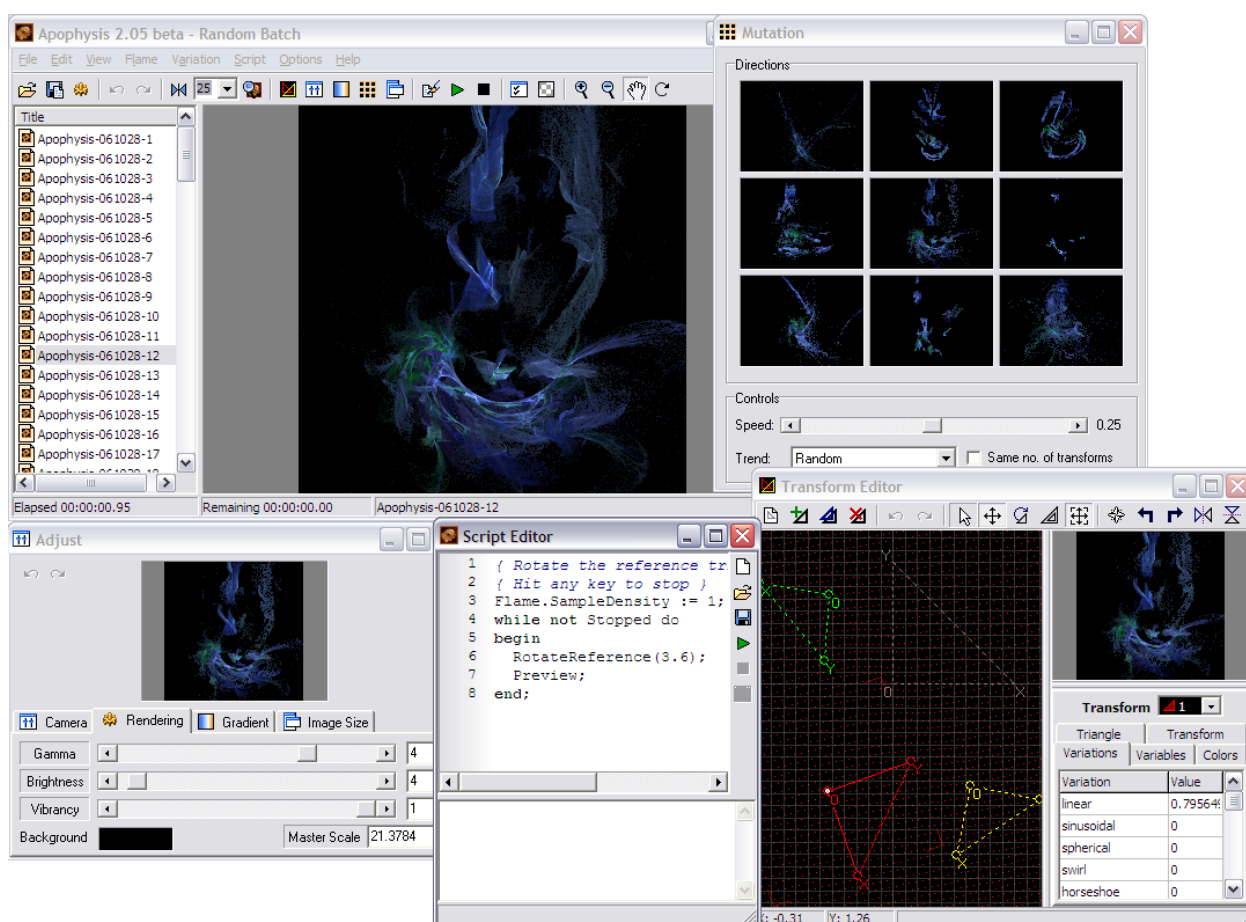


Рисунок 1 – Скриншот пользовательского интерфейса Apophysis с несколькими открытыми окнами

Apophysis — один из наиболее известных редакторов фрактальных пламен. Программа распространяется как open-source и предоставляет богатый набор инструментов для ручной настройки трансформаций, палитр и параметров рендера [3]. К достоинствам можно отнести простоту использования и обилие обучающих материалов. Основные ограничения связаны с настольным форматом, отсутствием встроенного хранения истории и невозможностью работы в распределённой среде.

Дополнительно стоит отметить, что основная работа в Apophysis строится вокруг ручного подбора вариаций и параметров, поэтому пользователь часто вынужден сохранять множество промежуточных файлов. Отсутствие центрального хранилища и единой учётной записи делает обмен результатами и совместную работу затруднительными.

1.2 JWildfire

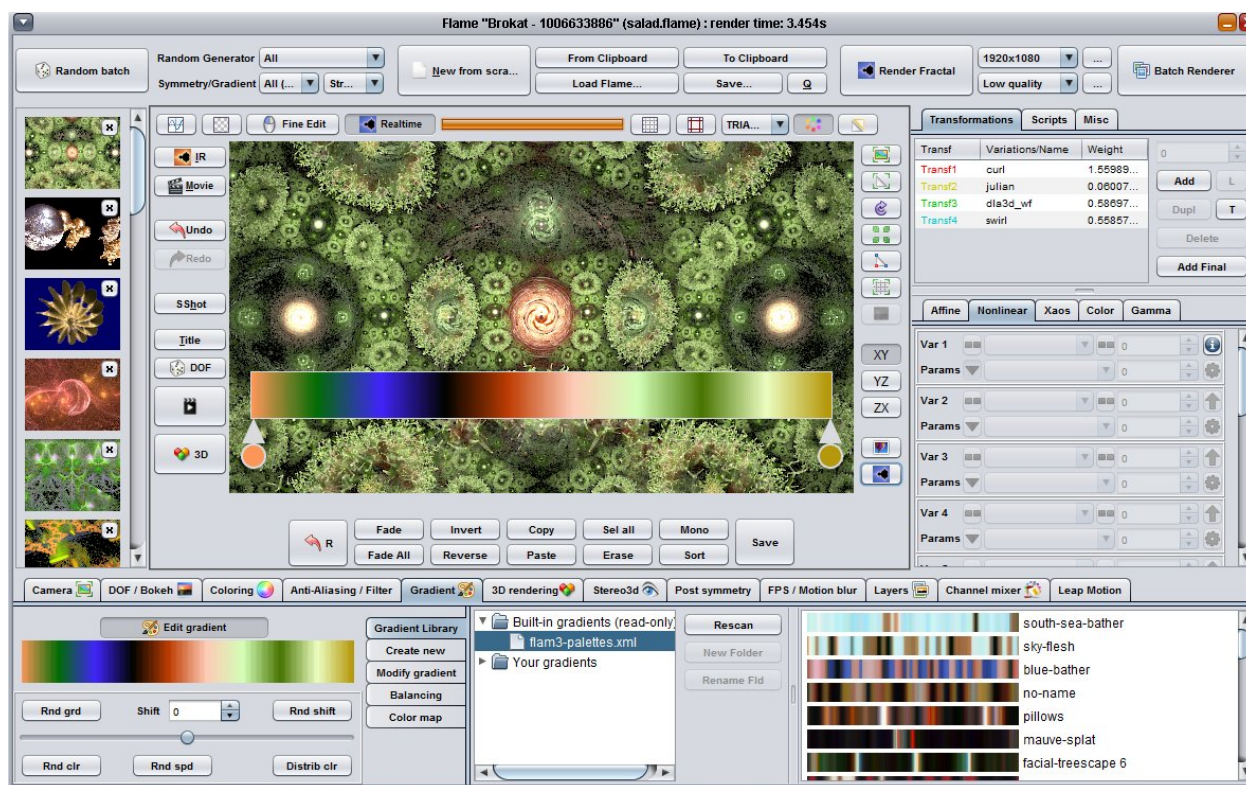


Рисунок 2 – Скриншот пользовательского интерфейса JWildfire

JWildfire — мощный кроссплатформенный редактор фрактальных пламен с большим набором вариаций и встроенными генераторами случайных фракталов [4]. Решение предоставляет широкие возможности, но требует значительного времени на освоение и ориентировано на опытных пользователей. Встроенные механизмы совместной работы и веб-публикации отсутствуют, а хранение результатов полностью локально.

Сильной стороной JWildfire является гибкость и глубина настроек, что делает его удобным для экспериментов и профессионального использования. Однако сложный интерфейс и необходимость локальной установки снижают доступность для начинающих пользователей и не решают задачу быстрого доступа к проектам с разных устройств.

1.3 Chaotica

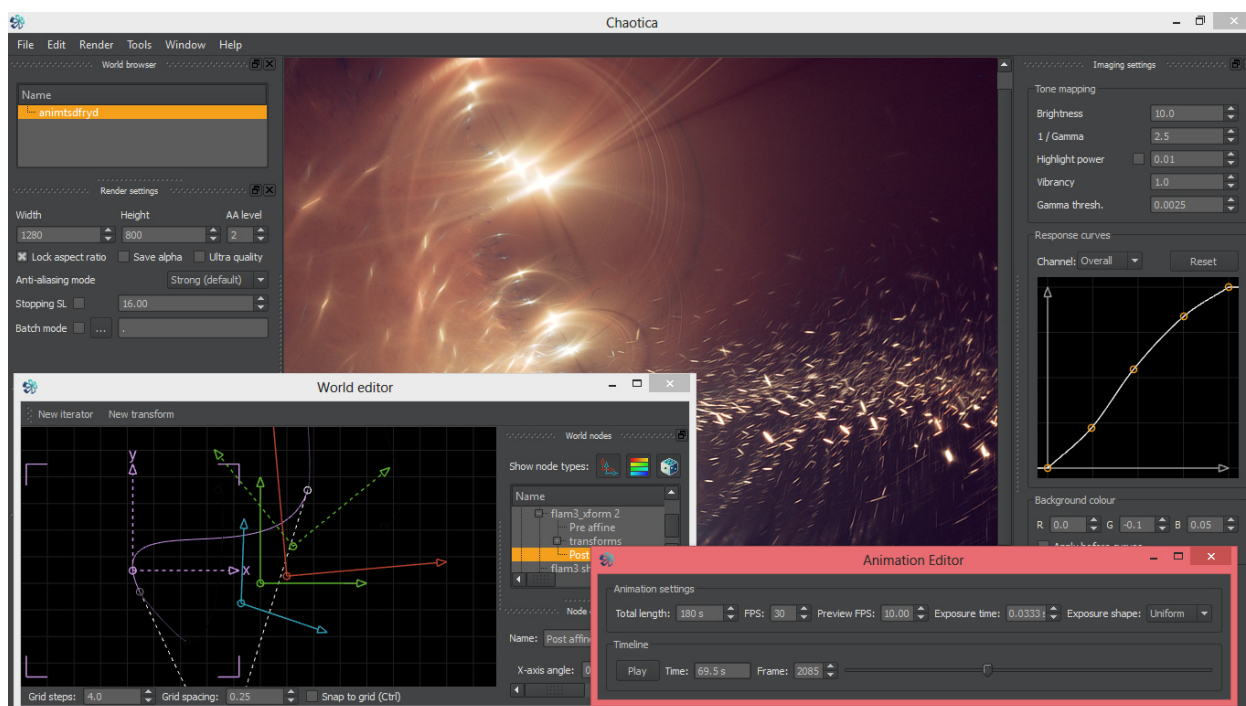


Рисунок 3 – Скриншот анимации фрактала, создаваемой в Chaotica

Chaotica — коммерческое приложение, ориентированное на получение высококачественных изображений и ускоренный рендеринг (включая GPU) [5]. Преимущества: высокая скорость и качество, развитый инструментарий анимации. Недостатки: закрытость, платная модель распространения и ориентация на локальную работу без интегрированного облачного хранения и пользовательских профилей.

В практической эксплуатации продукт удобен для студийных задач и финального качества рендера, однако ориентирован на автономное использование. Это ограничивает возможность модификаций, интеграции с веб-архивом и построения персонализированного профиля пользователя.

1.4 Сравнительный анализ

Таблица 1 – Сравнение аналогов по ключевым критериям

Продукт	Сильные стороны	Ограничения
Apophysis	Простота использования, open-source, богатая база примеров	Только desktop, нет истории и веб-доступа
JWildfire	Большое количество вариаций, кросс-платформенность, мощный набор функций	Высокая сложность интерфейса, локальное хранение, отсутствие веб-режима
Chaotica	Высокое качество рендера, поддержка GPU, развитая анимация	Платная лицензия, закрытость, отсутствие облачной истории

Анализ показал, что существующие решения преимущественно ориентированы на локальный рабочий процесс и не обеспечивают удобной работы с историей и публикацией результатов. Это определило требования к проектируемой системе, в частности необходимость веб-интерфейса, учётных записей и долговременного хранения рендеров.

Следовательно, ключевым конкурентным преимуществом будущей системы становится доступность из браузера и единое пространство проектов, где результаты сохраняются и доступны для повторного использования.

2 Формирование функциональных требований

На основе анализа аналогов сформированы функциональные требования к системе генерации фракталов:

- регистрация, вход и обновление пользовательской сессии с использованием JWT-токенов;
- создание задачи генерации фрактала с передачей размеров изображения, количества итераций, уровня симметрии, гамма-коррекции, набора функций и аффинных преобразований;
- получение списка доступных функций фрактального пламени;
- отображение прогресса генерации и промежуточного предпросмотра;

- получение списка ранее созданных фракталов с пагинацией, сортировкой и фильтрацией по пользователю;
 - просмотр отдельного фрактала, его параметров и итогового изображения;
 - отображение главной страницы с формой генерации и feed последних сгенерированных фракталов с подписями пользователей;
 - отображение страницы профиля с редактированием пользовательской информации и историей созданных фракталов;
 - сохранение параметров генерации в PostgreSQL и сохранение PNG-изображений в S3-совместимом хранилище MinIO;
 - редактирование базовых данных профиля пользователя.
- Сформированы также нефункциональные требования:
- производительность: приемлемое время рендера при стандартных параметрах качества;
 - масштабируемость: возможность независимо увеличивать количество экземпляров gateway и generator без изменения клиентского приложения;
 - надёжность: сохранность истории генераций в PostgreSQL и изображений в MinIO при перезапуске контейнеров;
 - безопасность: stateless-аутентификация через JWT, передача токена в HTTP-заголовке и проксирование его во внутренние gRPC-вызовы;
 - переносимость: развёртывание всех компонентов через Docker Compose;
 - сопровождаемость: разделение frontend, API gateway, IAM и generator по самостоятельным зонам ответственности;
 - ограничение вычислительной нагрузки: проверка максимальной ширины, высоты и числа итераций на стороне generator.

3 Анализ литературы

3.1 Принципы объектно-ориентированного проектирования

Объектно-ориентированный подход опирается на понятия инкапсуляции, наследования, полиморфизма и абстракции. Эти принципы позволяют моделировать предметную область в виде взаимосвязанных классов, обеспечивая повторное использование и расширяемость кода [6]. Для разрабатываемой системы это особенно важно, поскольку логика генерации фракталов включает множество разновидностей преобразований, параметров рендера и вариаций, которые удобно представлять в виде иерархий объектов.

В проекте объектные модели описывают параметры рендера, состояние проекта и пользовательский контекст, что позволяет разделить вычислительную и интерфейсную части. Это упрощает развитие

функциональности и делает код более устойчивым к изменениям.

3.2 Принципы SOLID и качество кода

Принципы SOLID ориентированы на снижение связности и повышение устойчивости к изменениям. В частности, разделение ответственности и открытость к расширению позволяют добавлять новые алгоритмы генерации и модули без переработки существующего кода [7]. Эти принципы были использованы при проектировании сервисов, чтобы обеспечивать независимость компонентов и удобство тестирования.

На практике это выразилось в выделении отдельных интерфейсов для рендера, хранения и пользовательских операций, а также в уменьшении числа прямых зависимостей между слоями. Такой подход упростил модульное тестирование и локализацию ошибок.

3.3 Шаблоны проектирования

В литературе по объектно-ориентированному проектированию выделяются шаблоны, упрощающие решение типовых инженерных задач [8]. Для проекта генерации фрактального пламени особенно важны шаблоны, которые позволяют отделить изменяемые алгоритмы от стабильной инфраструктуры приложения.

Шаблон Strategy применим к вариационным функциям фрактального пламени. Каждая функция преобразует точку по собственному правилу, но внешний код генерации может работать с общим интерфейсом функции. Благодаря этому добавление новой вариации не требует переписывать основной цикл рендера. Достаточно реализовать новую модель функции и зарегистрировать её в фабрике или строителе функций.

Шаблон Factory Method полезен при создании доменных объектов на основе входных DTO или protobuf-сообщений. Внешний контракт передаёт имя функции и числовые параметры, а внутри generator должен появиться объект конкретного класса. Вынесение этой логики из сервиса генерации снижает связность и делает ошибочные имена функций отдельным контролируемым сценарием.

Идея Builder применима к созданию задачи генерации. Задача включает размеры изображения, число итераций, симметрию, гамму, набор функций, набор аффинных преобразований и пользовательский контекст. Последовательная сборка такой конфигурации удобнее, чем передача большого количества несвязанных параметров между методами.

3.4 Моделирование предметной области

Для описания предметной области генерации фракталов применяются подходы доменно-ориентированного проектирования. Они позволяют

выделить ключевые сущности и описать их жизненный цикл [9]. В данном проекте основными сущностями являются пользователь, фрактал, вариационная функция, аффинное преобразование, задача генерации и изображение результата.

Сущность фрактала хранит параметры, достаточные для повторного отображения результата: размеры, количество итераций, уровень симметрии, гамму, список функций и список аффинных преобразований. Изображение при этом не смешивается с параметрами: в реляционной базе хранится только имя объекта, а бинарное содержимое помещается в объектное хранилище. Такое разделение поддерживает принцип единственной ответственности на уровне данных.

Пользовательская сущность относится к другому bounded context — IAM. Она не должна зависеть от деталей генерации фракталов. Поэтому generator хранит только внешний идентификатор пользователя, полученный из JWT, а полная информация о профиле остаётся в IAM. Это уменьшает связность микросервисов и упрощает изменение профиля без миграций в сервисе генерации.

3.5 Слоистая организация приложения

Серверные сервисы проекта разделяются на несколько слоёв: транспортный, сервисный, доменный и инфраструктурный. Транспортный слой принимает HTTP или gRPC-запросы и отвечает за преобразование данных из внешнего формата. Сервисный слой координирует сценарии использования: запуск генерации, получение истории, чтение изображения, обновление пользователя. Доменный слой содержит модели фрактала, функций и параметров. Инфраструктурный слой работает с PostgreSQL, MinIO и внешними библиотеками.

Такое разделение важно для тестируемости. Например, алгоритм генерации можно проверять отдельно от gRPC-сервера, а мапперы DTO можно тестировать без запуска базы данных. Кроме того, слой gateway не обязан знать внутреннее устройство generator: он опирается на контракт и выполняет роль адаптера между HTTP-клиентом и внутренними сервисами.

3.6 Контракты и переносимость моделей

В распределённой системе объектная модель существует сразу в нескольких формах: как доменная модель внутри сервиса, как схема базы данных, как protobuf-сообщение для gRPC и как JSON-модель во внешнем HTTP API. Если эти формы изменяются вручную и независимо, быстро появляются расхождения между клиентом и сервером.

Для снижения этого риска используются формальные контракты. gRPC-контракт generator описывает внутренние сообщения и сервисные

методы, а OpenAPI-спецификация описывает внешний HTTP API gateway. Кодогенерация по этим контрактам позволяет получать типы, интерфейсы и модели из единого источника правды. Это не отменяет ручное проектирование доменной модели, но делает границы между сервисами более проверяемыми.

4 Проектирование системы

4.1 Общая архитектура

После пересмотра проект реализуется как распределённая система из четырёх микросервисов:

- frontend — клиентское приложение на TypeScript, Vue, Tailwind CSS и Axios;
- API gateway — единая HTTP-точка входа для клиентского приложения, реализованная на Java и Spring;
- IAM — сервис управления пользователями и аутентификацией, реализованный на Go;
- generator — сервис генерации и хранения фракталов, реализованный на Java и Spring.

Frontend взаимодействует только с API gateway по HTTP. Gateway скрывает внутреннюю структуру backend-части и обращается к IAM и generator по gRPC [10]. Такой подход уменьшает связность клиентского приложения с внутренними сервисами: изменение gRPC-контрактов или схем хранения не требует изменения маршрутизации во frontend, если внешний HTTP API gateway остаётся совместимым.

Зоны ответственности сервисов представлены в таблице.

Таблица 2 – Ответственность компонентов системы

Компонент	Технологии	Ответственность
Frontend	TypeScript, Vue, Tailwind CSS, Axios	Отображение интерфейса, ввод параметров генерации, работа с токенами, получение событий прогресса и изображений через HTTP API gateway
API gateway	Java, Spring Boot, Spring Web, gRPC client	Приём HTTP-запросов, валидация внешнего API, преобразование REST/JSON-запросов в gRPC-вызовы IAM и generator, передача JWT во внутренние сервисы
IAM	Go, PostgreSQL, JWT	Регистрация, вход, обновление токенов, хранение пользователей и выдача access/refresh токенов
Generator	Java, Spring Boot, gRPC server, Spring Data JPA, Liquibase	Генерация фракталов, хранение параметров, выдача списка функций, поток прогресса задачи, сохранение и чтение изображений из MinIO

4.2 Внешний API и роль gateway

API gateway предоставляет frontend HTTP-интерфейс в формате JSON. HTTP-контракт описывается в OpenAPI-спецификации [11]. На стороне gateway по этой спецификации генерируются интерфейсы контроллеров и модели запросов/ответов, а на стороне frontend она используется как источник типизированных клиентов и DTO. Это уменьшает риск расхождения между клиентским кодом и серверным API.

По спецификации OpenAPI для работы с фракталами предусмотрены маршруты получения списка фракталов, получения фрактала по идентификатору, запуска генерации, подписки на события задачи, получения списка функций и получения PNG-изображения. Для пользовательских сценариев предусмотрены маршруты регистрации, входа, обновления токена, получения и изменения профиля.

Внутри gateway слой контроллеров отделён от слоя вызовов downstream-сервисов. Контроллер `FractalsController` принимает HTTP-запросы на маршрутах `/api/v1/fractals`, `/api/v1/fractals/gen`, `/api/v1/fractals/functions` и `/api/v1/fractals/images/{name}`. Сервисный слой использует gRPC-stub'ы generator: блокирующий stub применяется для коротких операций, а асинхронный stub — для подписки

на состояние задачи.

Отдельная роль gateway состоит в адаптации протоколов. Браузерное приложение работает с HTTP, JSON и text/event-stream, тогда как generator предоставляет gRPC API. Поэтому поток SubscribeToTask в generator преобразуется gateway в Server-Sent Events [12]. Это позволяет frontend получать прогресс рендера и путь к предпросмотру без прямого доступа к gRPC.

4.3 Внутренние gRPC-контракты

Основной внутренний контракт generator описан в файле `generator.proto`. Сервис Fractals предоставляет следующие операции:

- GetAll — получение списка фракталов с пагинацией, сортировкой и фильтрацией по `user_id`;
- GetById — получение одного фрактала по идентификатору;
- Generation — создание задачи генерации;
- SubscribeToTask — серверный поток состояния задачи;
- GetFunctions — получение доступных имён вариационных функций;
- GetImage — чтение изображения из S3-совместимого хранилища.

Запрос генерации содержит ширину и высоту изображения, число итераций, уровень симметрии, параметр гамма-коррекции, список вариационных функций и список аффинных преобразований. Ответ задачи содержит идентификатор фрактала, прогресс от 0 до 1 и имя файла предпросмотра или результата. Такое разделение удобно для асинхронного рендера: frontend быстро получает идентификатор задачи, а затем подписывается на события прогресса.

4.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс состоит из двух основных страниц. Главная страница объединяет форму генерации фрактала и feed последних работ. В форме пользователь задаёт размер изображения, количество итераций, уровень симметрии, гамму, вариационные функции и аффинные параметры. После запуска задачи на этой же странице отображается прогресс, промежуточный предпросмотр и итоговое изображение. Ниже располагается feed последних сгенерированных фракталов: для каждого элемента показываются изображение, основные параметры и подпись пользователя, создавшего работу.

Вторая страница отвечает за профиль пользователя. В верхней части размещается форма изменения пользовательской информации, включая имя и изображение профиля. Ниже отображается история фракталов текущего пользователя. Такое разделение поддерживает два основных сценария:

быстрый творческий цикл на главной странице и управление персональными данными вместе с архивом результатов на странице профиля.

4.5 Проектирование generator

Generator отвечает за вычислительно тяжёлую часть системы. Сервис принимает gRPC-запрос генерации, преобразует protobuf-модель во внутреннюю JPA-сущность, проверяет ограничения параметров и сохраняет базовую запись фрактала. Ограничения включают максимальную ширину, высоту и число итераций, а также обязательное наличие хотя бы одной вариационной функции и одного набора аффинных параметров.

После сохранения параметров создаётся задача генерации. Задача передаётся в сервис генерации, который выполняет рендер и публикует состояние выполнения. Для пользователя это выглядит как постепенное обновление прогресса, а на уровне backend реализуется через gRPC stream и последующее преобразование в SSE на gateway.

Сервис также содержит каталог поддерживаемых функций фрактального пламени. По коду generator доступны функции `linear`, `sinusoidal`, `spherical`, `swirl`, `horseshoe`, `polar`, `disc`, `heart` и `eyefish`. Каждая функция представлена отдельной моделью, что упрощает добавление новых вариаций без изменения внешнего API.

4.6 Проектирование IAM

IAM выделен в отдельный микросервис, чтобы логика пользователей и токенов не смешивалась с генерацией изображений. Сервис предоставляет внутренний gRPC API для регистрации, входа, обновления токена, получения пользователя и изменения профиля. Внешние HTTP-маршруты остаются на gateway, поэтому frontend не зависит от конкретной реализации IAM.

Аутентификация построена на JWT [13]. После успешного входа IAM выдаёт короткоживущий access token и refresh token. Access token используется frontend в заголовке `Authorization: Bearer token`. Gateway проверяет наличие JWT в HTTP-запросе и при обращении к generator или IAM передаёт токен дальше в gRPC metadata. Generator извлекает из токена идентификатор пользователя и сохраняет его в поле `user_id` у созданного фрактала.

4.7 Модель данных generator

Модель данных generator строится вокруг истории созданных фракталов. По миграциям Liquibase используются три таблицы: `fractals`, `functions` и `affine_params`. Таблица `fractals` хранит общие параметры результата: идентификатор, идентификатор пользователя, дату создания,

размеры изображения, число итераций, уровень симметрии, гамму и имя файла изображения. Таблица `functions` хранит набор вариационных функций и веса для конкретного фрактала. Таблица `affine_params` хранит цвет и коэффициенты аффинного преобразования a, b, c, d, e, f .

Связи имеют тип «один ко многим»: один фрактал содержит несколько функций и несколько наборов аффинных параметров. Для дочерних таблиц настроено каскадное удаление по внешнему ключу `fractal_id`, поэтому удаление фрактала не оставляет несвязанных параметров. Для ускорения выборки созданы индексы по `fractals.user_id`, `functions.fractal_id` и `affine_params.fractal_id`.

4.8 Модель данных IAM

Для IAM предусмотрена отдельная модель пользователей. В рамках текущих требований она состоит из одной таблицы `users`, где хранятся идентификатор пользователя, имя пользователя, хеш пароля и ссылка на изображение профиля или имя объекта в хранилище. Идентификатор пользователя используется как `subject` JWT и передаётся downstream-сервисам для связывания пользовательских действий с историей генераций.

4.9 Хранение изображений

Метаданные фракталов хранятся в PostgreSQL, а бинарные PNG-файлы вынесены в MinIO — локальное S3-совместимое объектное хранилище [14]. Такое разделение уменьшает нагрузку на реляционную базу данных и упрощает выдачу файлов. Generator использует AWS S3 SDK, создаёт `bucket images` при запуске, сохраняет PNG с типом `image/png` и возвращает имя файла, которое затем хранится в таблице `fractals`.

При запросе изображения `gateway` вызывает gRPC-метод `GetImage`; `generator` читает объект из MinIO и возвращает байтовое содержимое. Для `frontend` это остаётся обычным HTTP-ответом с изображением, а внутренняя реализация хранения скрыта за `gateway`.

4.10 Развёртывание

Развёртывание выполняется через Docker Compose [15]. Каждый сервис запускается в отдельном контейнере, а инфраструктурные зависимости поднимаются рядом с приложением. Для `generator` используются PostgreSQL и MinIO; контейнер `generator` получает адрес PostgreSQL через переменную `POSTGRES_URL`, адрес MinIO через `MINIO_URL`, а секрет JWT через `JWT_SIGNING_KEY`. `Gateway` получает адрес `generator` через `GENERATOR_URL`; аналогично для полной конфигурации задаётся адрес IAM.

Compose-сценарий содержит healthcheck'и для базы данных, MinIO, gateway и generator. Это снижает вероятность запуска приложения до готовности зависимостей и делает локальное окружение воспроизводимым.

Далее приведено краткое обоснование технологий, выбранных для реализации frontend, backend-сервисов, хранения данных и развёртывания системы.

4.11 Язык программирования Java



Рисунок 4 – Java

Язык программирования Java является объектно-ориентированным языком, разработанным компанией Sun Microsystems и в настоящее время поддерживаемым корпорацией Oracle. Java обеспечивает высокий уровень абстракции, платформенную независимость и безопасность, что делает его одним из самых популярных языков в мире.

Одним из основных преимуществ Java является его независимость от платформы, достигаемая благодаря использованию виртуальной машины Java (JVM), которая исполняет байт-код, одинаковый для всех операционных систем. Это свойство позволяет создавать переносимые приложения без необходимости модификации исходного кода.

Java поддерживает основные принципы объектно-ориентированного программирования, такие как наследование, инкапсуляция, полиморфизм и абстракция. Эти особенности обеспечивают высокую модульность и повторное использование кода. [16]

В рамках проекта Java используется в двух сервисах: API gateway и generator. Для этих компонентов язык выбран благодаря следующим преимуществам:

- наличие развитой экосистемы для HTTP API, gRPC, работы с базами данных и контейнеризированного запуска;
- развитая экосистема, включающая библиотеки и фреймворки, ускоряющие процесс разработки;

- встроенная система управления памятью с автоматическим сборщиком мусора, упрощающая разработку и снижающая вероятность ошибок, связанных с управлением ресурсами;

- высокая читаемость и строгая типизация, что способствует написанию надёжного и легко сопровождаемого кода.

В gateway Java применяется для обработки HTTP-запросов, настройки фильтра JWT и вызова gRPC-сервисов. В generator язык используется для реализации алгоритма фрактального пламени, очереди задач, сохранения метаданных и работы с S3-совместимым хранилищем.

4.12 Фреймворк Spring



Рисунок 5 – Spring Framework

Spring — это мощный и гибкий фреймворк с открытым исходным кодом для разработки корпоративных и веб-приложений на языке Java. Благодаря своей модульной структуре, высокой производительности и удобству масштабирования, Spring широко применяется при создании распределённых и высоконагруженных систем, включая веб-сервисы и многопользовательские онлайн-платформы.

Одной из ключевых особенностей Spring является внедрение зависимостей (Dependency Injection), которое способствует слабой связанности компонентов приложения и облегчает их тестирование, повторное использование и расширение. Фреймворк построен по принципу инверсии управления (Inversion of Control), где управление жизненным циклом объектов осуществляется специальным контейнером.

В проекте Spring используется в gateway и generator. Gateway принимает HTTP-запросы от frontend, а generator предоставляет gRPC API для генерации фракталов и доступа к истории.

В процессе разработки были использованы следующие модули Spring:

- Spring Boot — модуль, предназначенный для упрощения конфигурации и быстрой инициализации проекта. Он предоставляет встроенный сервер и автоматическую настройку компонентов, что позволило сосредоточиться на реализации логики приложения;

- Spring Web — модуль, реализующий обработку HTTP-запросов, маршрутизацию и построение REST API gateway;

- Spring gRPC — модуль для реализации gRPC-сервера в generator и gRPC-клиента в gateway;
- Spring Data JPA — модуль для работы с PostgreSQL через объектно-реляционное отображение;
- Spring Security — модуль, обеспечивающий обработку JWT в HTTP-запросах gateway и gRPC metadata generator.

Использование Spring позволило добиться высокой скорости обработки запросов, надёжности и масштабируемости серверной части. Благодаря модульной архитектуре фреймворка, разработка велась по принципу разделения ответственности, что улучшило структуру проекта и упростило поддержку и развитие функционала. [17]

Дополнительно применяются встроенные механизмы валидации данных, централизованная обработка ошибок и healthcheck'и, что обеспечивает единый формат ответов и предсказуемость поведения сервисов.

4.13 Язык Go

Язык Go используется для реализации IAM-сервиса. Для сервиса аутентификации важны быстрый запуск, простая модель конкурентного выполнения, компактная поставка и удобная работа с сетевыми сервисами. Go хорошо подходит для небольших backend-компонентов, которые предоставляют API, работают с базой данных и не требуют тяжёлой runtime-среды [18].

В рамках проекта IAM отвечает за регистрацию пользователей, проверку пароля, выпуск JWT и обновление сессии. Выделение этого сервиса на Go позволяет развивать пользовательскую часть независимо от Java-сервисов генерации.

4.14 Frontend: TypeScript, Vue, Tailwind CSS и Axios

Frontend реализуется как одностраничное веб-приложение. TypeScript добавляет статическую типизацию поверх JavaScript и помогает согласовать модели запросов и ответов с OpenAPI-спецификацией [19]. На основе OpenAPI-контракта формируются типы и клиентские функции для обращения к gateway, поэтому изменения HTTP API быстрее обнаруживаются на этапе сборки frontend. Vue выбран как фреймворк для построения реактивного интерфейса: он упрощает разбиение экрана на компоненты, управление состоянием формы генерации и обновление истории фракталов [20].

Tailwind CSS применяется для быстрой сборки адаптивного интерфейса из утилитарных классов [21]. Такой подход удобен для страниц с большим количеством параметров генерации: размеры, отступы, состояния кнопок и сетки можно задавать непосредственно рядом с

разметкой компонента.

Axios используется как HTTP-клиент для обращений frontend к API gateway [22]. Через него выполняются запросы регистрации и входа, получение списка фракталов, запуск генерации и загрузка изображений. Для прогресса рендера применяется поток событий `text/event-stream`.

4.15 Система сборки Gradle

Gradle используется для сборки Java-сервисов gateway и generator. Он обеспечивает управление зависимостями, генерацию Java-кода из protobuf-контрактов, запуск тестов и сборку исполняемых приложений. В gateway дополнительно подключён OpenAPI Generator: из файла `openapi.yaml` формируются Java-модели и API-интерфейсы, которые затем реализуются контроллерами сервиса [23].

4.16 База данных PostgreSQL



Рисунок 6 – PostgreSQL

Для хранения структурированных данных используется система управления базами данных PostgreSQL [24]. Она выбрана из-за поддержки транзакций, ограничений целостности, индексов и зрелой интеграции с backend-стеком. Конкретная схема данных generator и IAM описана выше в подразделах о моделях данных, поэтому в данном месте фиксируется только обоснование выбора СУБД.

4.17 MinIO и S3-совместимое хранилище

Для хранения изображений используется MinIO — S3-совместимое объектное хранилище [14]. Оно позволяет отделить бинарные PNG-файлы от реляционных данных и оставить PostgreSQL для метаданных, связей и поиска. Благодаря совместимости с S3 при необходимости локальное

хранилище можно заменить внешним объектным хранилищем без изменения пользовательских сценариев.

4.18 gRPC

gRPC применяется для внутреннего взаимодействия gateway с generator и IAM [10]. По сравнению с передачей JSON между backend-сервисами protobuf-контракты дают строгую схему сообщений, компактное представление данных и удобную генерацию клиентского и серверного кода. Подробный сценарий потоковой передачи прогресса рассмотрен в подразделе о внутренних gRPC-контрактах.

4.19 JWT

JSON Web Token используется как формат access token [13]. Токен содержит идентификатор пользователя и имя, подписывается секретным ключом и проверяется без обращения к базе данных при каждом запросе. Благодаря этому gateway остаётся stateless, а механизм передачи пользовательского контекста между сервисами не требует общей HTTP-сессии.

4.20 Контейнеризация Docker и Docker Compose

Для развёртывания использовалась контейнеризация Docker. Каждый микросервис упакован в отдельный образ, а общий сценарий запуска описан в Docker Compose [15]. Compose выбран как достаточный для курсового проекта способ поднять приложение, базу данных и MinIO одной командой в воспроизводимом локальном или серверном окружении.

4.21 Среда разработки Visual Studio Code

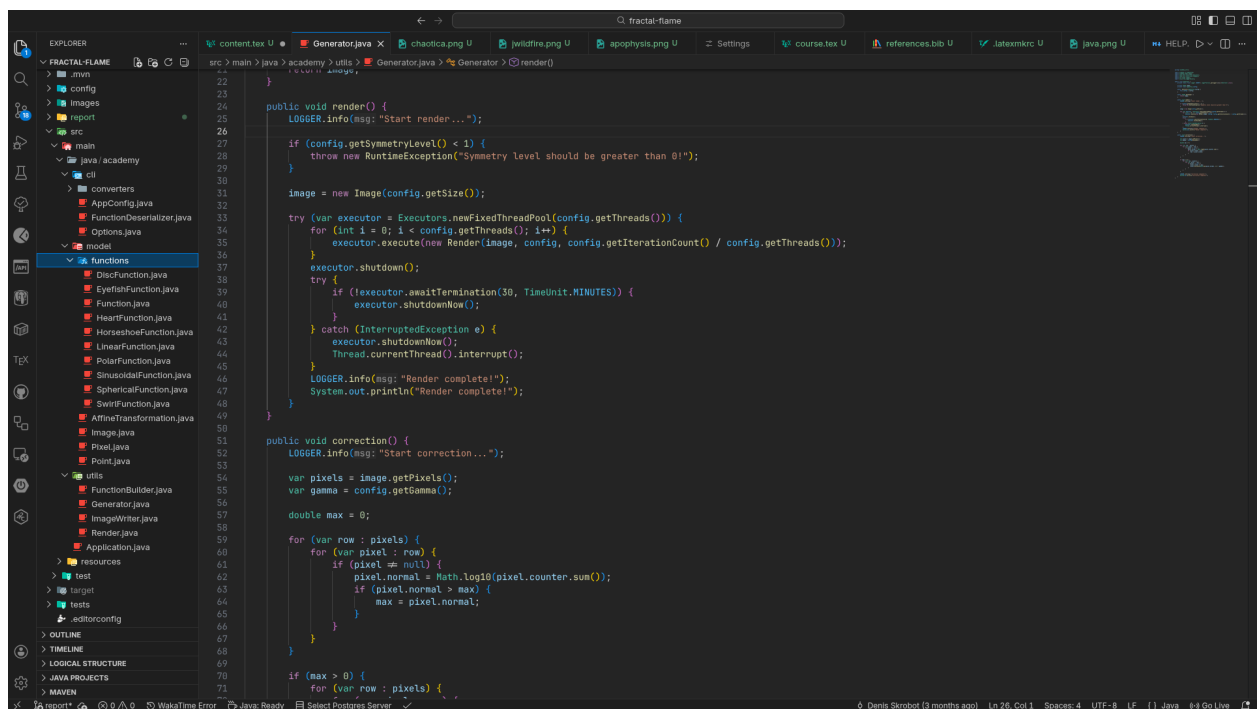


Рисунок 7 – Экран редактирования кода Visual Studio Code

Visual Studio Code (VS Code) — это лёгкая, но мощная интегрированная среда разработки, разработанная компанией Microsoft. С момента своего выхода в 2015 году она завоевала огромную популярность благодаря простоте, скорости и гибкости. VS Code является кроссплатформенным инструментом, доступным на Windows, macOS и Linux, что делает его универсальным решением для разработчиков по всему миру. [25]

VS Code не перегружена лишними функциями, но в то же время предоставляет всё необходимое для разработки: интеллектуальную подсветку синтаксиса, автодополнение, отладку, поддержку Git, систему контроля версий и встроенный терминал. Это позволяет разрабатывать и тестировать код быстро и эффективно, без лишних задержек.

Основные причины выбора VS Code включают:

1. Лёгкость и быстродействие. VS Code быстро запускается и работает с минимальными задержками, что делает её идеальной для разработки фронтенд-части и быстрого прототипирования.
2. Расширяемость. Благодаря множеству доступных расширений, VS Code легко адаптируется под любые задачи.
3. Интеграция с Git и другими системами контроля версий. VS Code имеет встроенную поддержку Git, что упрощает работу с репозиториями и позволяет эффективно управлять версиями проекта.

Разработка всех компонентов проекта выполнялась исключительно в

Visual Studio Code, что обеспечило единообразие среды и повторяемость процесса. Дополнительно использовались расширения для Java, Docker и работы с $\text{\LaTeX}$, что позволило вести и разработку, и оформление документации в одном инструменте. Даже этот документ написан в VS Code с использованием $\text{\LaTeX}$.

4.22 Хостинг и развёртывание приложения

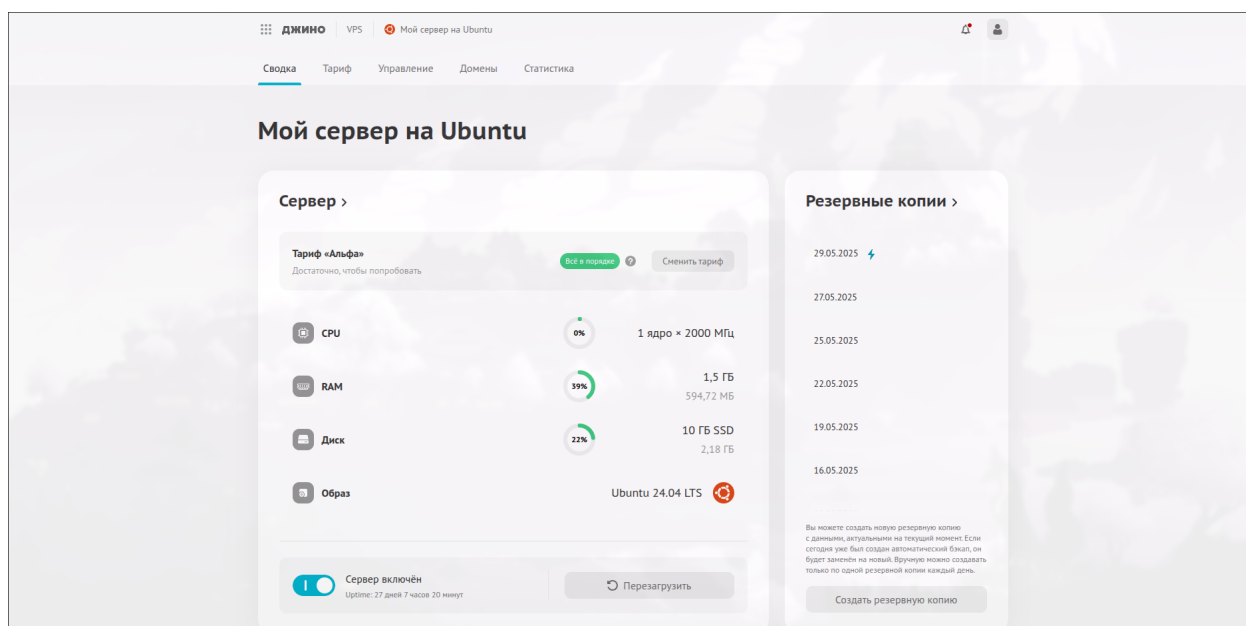


Рисунок 8 – Страница сервера на jino.ru

Финальная версия системы разворачивается в Docker-контейнерах на VPS; ниже приведён общий сценарий размещения и администрирования.

Для размещения серверной части проекта использовался платный хостинг-провайдер Jino.ru. Выбор данного провайдера был обусловлен возможностью полного управления сервером через SSH-доступ, поддержкой системных сервисов, а также интеграцией доменов и SSL-сертификатов. [26]

Развёртывание приложения осуществляется на выделенный виртуальный сервер (VPS), работающий под управлением операционной системы Linux. Доступ к серверу выполняется с использованием безопасного протокола SSH, что позволяет выполнять все действия удалённо: загрузку файлов, настройку служб и управление серверной частью.

В составе развёртывания применяется обратный прокси для маршрутизации запросов и разграничения доступа к сервисам, а также централизованное логирование. Это упростило администрирование и позволило быстро выявлять ошибки в работе компонентов.

На сервере запускается тот же Docker Compose-проект, что используется при локальной проверке. Отличаются только значения

переменных окружения, доменное имя и настройки обратного прокси. Такой вариант развёртывания проще ручной установки отдельных JAR-файлов и снижает риск расхождения между средой разработки и сервером.

Compose-конфигурация обеспечивает:

- воспроизводимый запуск всех компонентов;
- настройку зависимостей между сервисами и healthcheck'ов;
- перезапуск контейнеров в случае сбоя;
- изоляцию внутренних gRPC-портов от внешнего доступа.

Для доступа к приложению по HTTPS был зарегистрирован собственный домен, приобретённый через панель управления Jino. К домену подключён wildcard-сертификат, позволяющий защищать не только основной домен, но и все его поддомены. Сертификат установлен на стороне веб-сервера, который при необходимости может проксировать запросы к приложению, обеспечивая безопасную и надёжную маршрутизацию трафика.

Использование Jino в качестве хостинг-платформы позволило обеспечить:

- полную автономность и контроль над серверной средой;
- надёжную работу приложения в режиме 24/7;
- безопасный доступ пользователей по защищённому протоколу HTTPS;
- удобную интеграцию домена и сертификатов.

Развёртывание проекта в реальной среде подтвердило его работоспособность вне среды разработки и позволило провести полноценное тестирование функциональности в условиях, приближенных к промышленной эксплуатации.

5 Тестирование программного обеспечения

5.1 Стратегия тестирования

Тестирование системы проводилось на нескольких уровнях: модульном, интеграционном и системном. На модульном уровне проверялась корректность отдельных компонентов генерации и обработки параметров. Интеграционные тесты подтверждали корректность взаимодействия микросервисов, а системные тесты моделировали реальную работу пользователя. Подход соответствует классическим рекомендациям по организации тестирования программного обеспечения [27].

Для проверки критических сценариев использовались контрольные наборы параметров рендера и заранее подготовленные эталонные результаты. Это позволило выявлять регрессии при изменениях алгоритмов и конфигурации.

5.2 Сценарии тестирования

Ниже приведены основные сценарии, подтверждающие работоспособность системы:

1. Регистрация и вход пользователя: создание записи в IAM, получение access token и refresh token.
2. Проверка JWT: передача access token из frontend в gateway и дальнейшая передача токена во внутренний gRPC-вызов.
3. Создание задачи генерации: отправка параметров фрактала через HTTP API gateway и получение идентификатора задачи.
4. Подписка на прогресс: получение событий text/event-stream, сформированных gateway на основе gRPC stream generator.
5. Сохранение результата: запись параметров фрактала в PostgreSQL и PNG-изображения в MinIO.
6. Просмотр истории: получение списка фракталов с пагинацией, сортировкой и фильтрацией по пользователю.
7. Получение изображения: запрос изображения через gateway и чтение объекта из MinIO сервисом generator.
8. Проверка ограничений параметров: отказ generator при превышении максимальных размеров или количества итераций.

Результаты тестирования подтвердили корректность работы ключевых функций и соответствие заявленным требованиям.

Отдельно фиксировались временные показатели рендера и устойчивость к отказам отдельных сервисов, что подтвердило работоспособность системы в условиях повышенной нагрузки.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была спроектирована система генерации изображений фрактального пламени. Работа началась с анализа аналогов: Apophysis, JWildfire и Chaotica. Рассмотренные решения обладают развитыми инструментами настройки фракталов, однако в основном ориентированы на локальное использование. Это позволило сформулировать основную идею проекта: предоставить пользователю веб-интерфейс, историю созданных работ, хранение результатов и доступ к генерации без установки специализированного настольного приложения.

На основе анализа были сформированы функциональные и нефункциональные требования. К ключевым функциям относятся регистрация и вход пользователя, запуск генерации фрактала, получение прогресса задачи, просмотр последних работ, работа с профилем и просмотр персональной истории. Нефункциональные требования включают переносимость окружения, разделение ответственности между сервисами, возможность масштабирования вычислительной части и сохранность данных при перезапуске контейнеров.

В результате проектирования была выбрана микросервисная архитектура из четырёх компонентов: frontend, API gateway, IAM и generator. Frontend взаимодействует с gateway по HTTP, а gateway обращается к IAM и generator по gRPC. Такое решение отделяет внешний клиентский контракт от внутренних сервисов и позволяет независимо развивать пользовательский интерфейс, аутентификацию и вычислительный контур. Для внешнего HTTP API используется OpenAPI-спецификация, а для внутренних вызовов generator — protobuf-контракт gRPC.

Отдельно была проработана модель хранения данных. В generator параметры фракталов сохраняются в PostgreSQL в таблицах `fractals`, `functions` и `affine_params`. Изображения вынесены в MinIO как S3-совместимое объектное хранилище, что разгружает реляционную базу и упрощает выдачу PNG-файлов. IAM хранит пользовательские данные отдельно, а связь между пользователем и созданными фракталами выполняется через идентификатор пользователя из JWT.

Выбранный стек соответствует задачам проекта. Java и Spring используются для gateway и generator, Go — для IAM, TypeScript, Vue, Tailwind CSS и Axios — для frontend. Docker Compose обеспечивает единый сценарий запуска приложения и инфраструктурных зависимостей. Кодогенерация по OpenAPI и protobuf снижает вероятность расхождений между контрактами и реализацией, а Server-Sent Events позволяют передавать прогресс генерации в браузер без прямого использования gRPC на frontend.

Проведённое проектирование показывает, что система может развиваться дальше без радикального пересмотра архитектуры. В качестве

направлений развития можно выделить расширение набора вариационных функций, добавление очереди фоновых задач, более гибкий редактор аффинных преобразований, улучшение страницы профиля, внедрение ролей пользователей и подготовку финальных диаграмм для документации. Также перспективным является вынесение генерации в масштабируемый пул worker-процессов, если нагрузка на рендеринг станет выше.

Таким образом, цель курсового проекта достигнута на уровне проектирования и реализации ключевых инженерных решений. Полученная архитектура поддерживает основные пользовательские сценарии, соответствует требованиям предметной области и создаёт основу для дальнейшей разработки полноценного веб-приложения для генерации фрактального пламени.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Draves, S. The Fractal Flame Algorithm / S. Draves, E. Reckase // The Flame Algorithm. — 2003. — 41 P.
- [2] Newman, Sam. Building Microservices / Sam Newman. — 2 edition. — O'Reilly Media, 2021.
- [3] Apophysis: Fractal Flame Editor [Электронный ресурс]. — Режим доступа : <https://sourceforge.net/projects/apophysis/>.
- [4] JWildfire: Fractal Flame Generator (Repository) [Электронный ресурс]. — Режим доступа : <https://github.com/thargor6/JWildfire>.
- [5] Chaotica Fractal Flame Software [Электронный ресурс]. — Режим доступа : <https://www.chaoticafractals.com/>.
- [6] Booch, G. Object-Oriented Analysis and Design with Applications / G. Booch. — 3 edition. — Addison-Wesley Professional, 2007.
- [7] Martin, Robert C. Agile Software Development: Principles, Patterns, and Practices / Robert C. Martin. — Prentice Hall, 2002.
- [8] Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma, R. Helm, R. Johnson, J. Vlissides. — Addison-Wesley, 1994.
- [9] Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Addison-Wesley, 2003.
- [10] gRPC Documentation [Электронный ресурс]. — Режим доступа : <https://grpc.io/docs/>.
- [11] OpenAPI Specification [Электронный ресурс]. — Режим доступа : <https://spec.openapis.org/oas/latest.html>.
- [12] Server-sent events — MDN Web Docs [Электронный ресурс]. — Режим доступа : https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events.
- [13] Jones, M. JSON Web Token (JWT): Tech. Rep. RFC 7519 / M. Jones, J. Bradley, N. Sakimura: IETF, 2015. — Режим доступа : <https://www.rfc-editor.org/rfc/rfc7519>.
- [14] MinIO Object Store Documentation [Электронный ресурс]. — Режим доступа : <https://min.io/docs/minio/linux/index.html>.
- [15] Docker Compose Documentation [Электронный ресурс]. — Режим доступа : <https://docs.docker.com/compose/>.
- [16] Schildt, H. Java: The Complete Reference, Ninth Edition / H. Schildt. — McGraw-Hill Education, 2014. — Pp. 17 – 23.
- [17] Walls, Craig. Spring in Action / Craig Walls. — 6 edition. — Manning Publications, 2022.
- [18] The Go Programming Language Documentation [Электронный ресурс]. — Режим доступа : <https://go.dev/doc/>.
- [19] TypeScript Documentation [Электронный ресурс]. — Режим доступа : <https://www.typescriptlang.org/docs/>.
- [20] Vue.js Documentation [Электронный ресурс]. — Режим доступа : <https://vuejs.org/>.

- [//vuejs.org/guide/introduction.html](https://vuejs.org/guide/introduction.html).
- [21] Tailwind CSS Documentation [Электронный ресурс]. — Режим доступа : <https://tailwindcss.com/docs>.
 - [22] Axios Documentation [Электронный ресурс]. — Режим доступа : <https://axios-http.com/docs/intro>.
 - [23] Gradle User Manual [Электронный ресурс]. — Режим доступа : <https://docs.gradle.org/current/userguide/userguide.html>.
 - [24] PostgreSQL Documentation [Электронный ресурс]. — Режим доступа : <https://www.postgresql.org/docs/>.
 - [25] Visual Studio Code Documentation [Электронный ресурс]. — Режим доступа : <https://code.visualstudio.com/docs>.
 - [26] Jino.ru: виртуальный хостинг и VPS [Электронный ресурс]. — Режим доступа : <https://jino.ru/>.
 - [27] Myers, Glenford J. The Art of Software Testing / Glenford J. Myers, Corey Sandler, Tom Badgett. — 3 edition. — Wiley, 2011.
 - [28] Robbins, Jennifer Niederst. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics / Jennifer Niederst Robbins. — 5 edition. — O'Reilly Media, 2018.